

Constructors

1. Write a class with a default constructor, one argument constructor and two argument constructors. Instantiate the class to call all the constructors of that class from a main class

Answer:

Student.java

```
class Student {  
    Student() {  
        System.out.println("Default Constructor");  
    }  
    Student(int id) {  
        System.out.println("One Argument Constructor");  
        System.out.println("ID: " + id);  
    }  
    Student(int id, String name) {  
        System.out.println("Two Argument Constructor");  
        System.out.println("ID: " + id);  
        System.out.println("Name: " + name);  
    }  
}
```

Main.java

```
public class Main {  
    public static void main(String[] args) {  
        Student s1 = new Student();  
        Student s2 = new Student(101);  
        Student s3 = new Student(102, "Naveen");  
    }  
}
```

Output:

Default Constructor

One Argument Constructor

ID: 101

Two Argument Constructor

ID: 102

Name: Naveen

2. Call the constructors(both default and argument constructors) of super class from a child class

Answer:

```
class Parent {  
    Parent() {  
        System.out.println("Parent Default Constructor");  
    }  
    Parent(int x) {  
        System.out.println("Parent Argument Constructor: " + x);  
    }  
}  
  
class Child extends Parent {  
    Child() {  
        super();  
        System.out.println("Child Default Constructor");  
    }  
    Child(int x) {  
        super(x);  
        System.out.println("Child Argument Constructor");  
    }  
    public static void main(String[] args) {
```

```
        new Child();  
        new Child(100);  
    }  
}
```

Output:

Parent Default Constructor

Child Default Constructor

Parent Argument Constructor: 100

Child Argument Constructor

3. Apply private, public, protected and default access modifiers to the constructor

Answer:

```
class Demo {  
    public Demo() {  
        System.out.println("Public Constructor");  
    }  
    protected Demo(int a) {  
        System.out.println("Protected Constructor");  
    }  
    Demo(String s) {  
        System.out.println("Default Constructor");  
    }  
    private Demo(double d) {  
        System.out.println("Private Constructor");  
    }  
    public static void main(String[] args) {  
        new Demo();  
        new Demo(10);  
    }  
}
```

```
        new Demo("Java");  
        new Demo(10.5); // Allowed because it's inside the same class  
    }  
}
```

Output:

Public Constructor

Protected Constructor

Default Constructor

Private Constructor

4. Write constructors with return type int and String

Answer:

Not Possible in Java.

A constructor **cannot have any return type**, not even void.

Invalid example:

```
class Demo {  
    int Demo() {                // Not a constructor  
        return 10;  
    }  
    String Demo() {            // Compile-time error  
        return "Hello";  
    }  
}
```

5. Try to call the constructor multiple times with the same object

Answer:

Not Possible in Java.

A constructor is called **only once** when an object is created using new.

```
class Demo {  
    Demo() {  
        System.out.println("Constructor Called");  
    }  
    public static void main(String[] args) {  
        Demo obj = new Demo();  
        // obj.Demo(); Not allowed  
    }  
}
```

To call the constructor again, you must create a **new object**:

```
Demo obj1 = new Demo();
```

```
Demo obj2 = new Demo();
```

Output:

Constructor Called

Constructor Called